Introduction to GenAI

Complete Quick-Revision Notes + MCQ Bank

Covers: ML Foundations · Data Prep · Models · Evaluation · Trees · NNs · NLP · LLMs

1 1

1.1 1

ML vs Traditional Programming

Core Idea

Traditional: Rules + Data → Output
ML: Data + Output → Rules (learned)

1.2 1

AI vs ML vs DL

- **AI**: Broad field — machines mimicking human intelligence
- **ML**: Subset of AI; learns from data without explicit rules
- **DL**: Subset of ML; uses deep neural networks (many layers)

Hierarchy: AI ⊃ ML ⊃ DL

1.3 1

Types of Learning

- **Supervised**: Labeled data; learns input→output mapping (e.g., spam detection)
- **Unsupervised**: No labels; finds hidden patterns (e.g., clustering)
- **Semi-Supervised**: Mix of labeled + unlabeled data
- **Reinforcement**: Agent learns by reward/penalty from environment (e.g., game AI)

1.4 1

Types of ML Tasks

Classification

Predict a **category**
e.g., Email = spam/ham
Output: discrete class

Regression

Predict a **number**
e.g., House price
Output: continuous value

Clustering

Group similar items
e.g., Customer segments
Output: cluster labels

2 1

2.1 1

Handling Missing Values

- **Drop rows/cols**: if too many missing
- **Mean/Median imputation**: numerical (median if skewed)
- **Mode imputation**: categorical
- **Forward/Backward fill**: time-series
- **Model-based**: use another ML model to predict missing

2.2 1

Feature Scaling

Min-Max Normalization (0–1):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Z-Score Standardization (mean=0, std=1):

$$x' = \frac{x - \mu}{\sigma}$$

Use *standardization* for algorithms sensitive to outliers (SVM, PCA); *normalization* for NNs.

2.3 1

Data Encoding Methods

- **Label Encoding**: Assigns integer to each category (0, 1, 2...). Risk: implies order.
- **One-Hot Encoding**: Binary column per category. No ordinal assumption. Curse of dimensionality with high cardinality.
- **Ordinal Encoding**: Like label but with meaningful order (S < M < L).
- **Target Encoding**: Replace category with mean of target variable.

Python: Encoding

```python
from sklearn.preprocessing import (
  LabelEncoder, OneHotEncoder,
  StandardScaler, MinMaxScaler)

le = LabelEncoder()
y = le.fit_transform(['cat','dog','cat'])

ohe = OneHotEncoder(sparse=False)
X = ohe.fit_transform(df[['color']])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3 1

3.1 1

Linear Regression

Model

$$\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \boldsymbol{\theta}^T \mathbf{x}$$

Cost (MSE):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

Normal Equation:

$$\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$$

Gradient Descent:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Assumptions (LINE):

1. **L**inearity: linear relationship between X and y
2. **I**ndependence: observations independent
3. **N**ormality: residuals normally distributed
4. **E**qual variance (Homoscedasticity): constant variance of errors
5. No multicollinearity among features

Limitations:

- Sensitive to outliers
- Assumes linear relationship
- Multicollinearity inflates coefficients
- Not for classification

Linear Regression (sklearn)

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"MSE: {mean_squared_error(y_test, y_pred):.3f}, R2: {r2_score(y_test, y_pred):.3f}"
    )
print(f"Coefficients: {model.coef_}, Intercept: {model.intercept_:.3f}")
```

3.2 1

Logistic Regression

Sigmoid & Model

Sigmoid:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad z = \boldsymbol{\theta}^T \mathbf{x}$$

Range: $(0, 1)$; $\sigma(0) = 0.5$

Prediction: $\hat{y} = 1$ if $\hat{p} \geq 0.5$

Log-Loss (Binary Cross-Entropy):

$$J = -\frac{1}{m} \sum [y \log \hat{p} + (1-y) \log(1-\hat{p})]$$

Assumptions:

- Binary outcome (for binary LR)
- Linearity of log-odds w.r.t. features
- No multicollinearity
- Large sample size preferred

Limitations:

- Only linear decision boundary
- Sensitive to outliers
- Requires feature engineering for complex patterns
- Assumes features are independent

Logistic Regression (sklearn)

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
print(f"Probabilities: {clf.predict_proba(X_test[:2])}")
```

4 1

4.1 1

Bias & Variance

Decomposition

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

- **High Bias** = Underfitting (model too simple)
- **High Variance** = Overfitting (model too complex)
- **Tradeoff**: Increasing model complexity ↓ bias, ↑ variance
- **Fix Underfitting**: More features, complex model, less regularization
- **Fix Overfitting**: More data, regularization (L1/L2), dropout, pruning

4.2 1

Confusion Matrix

	Pred +	**Pred -**
Actual +	TP	FN
Actual -	FP	TN

Classification Metrics

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP+FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

When to use what: Precision matters when FP is costly (spam). Recall matters when FN is costly (cancer detection). F1 is balance.

4.3 1

Regression Metrics

Regression Metrics

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$\text{Adj-}R^2 = 1 - (1-R^2)\frac{n-1}{n-p-1}$$

MAE: robust to outliers; *MSE/RMSE*: penalizes large errors; R^2: proportion of variance explained (1=perfect); Adj-R^2 penalizes extra features.

4.4 1
Cross-Validation

k-Fold CV: Split data into k folds; train on $k-1$, test on 1; repeat k times; average scores.

Stratified k-Fold: Preserves class distribution per fold. Use for imbalanced datasets.

Cross-Validation

```
from sklearn.model_selection import (
  cross_val_score, StratifiedKFold)

cv = StratifiedKFold(n_splits=5)
scores = cross_val_score(
  model, X, y, cv=cv,
  scoring='f1_macro')
print(f"CV F1: {scores.mean():.3f}"
      f" +/- {scores.std():.3f}")
```

5 1

5.1 1
Introduction

A tree-like model of decisions. Each **internal node** = feature test; **branch** = outcome; **leaf** = prediction.

Algorithm: Recursively split on the feature that gives best information gain (or lowest impurity).

5.2 1
Gini Impurity

Gini

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k = fraction of class k at node t.
Pure node: Gini = 0. Max impurity (equal split): Gini = 0.5 (binary).

5.3 1
Information Gain (Entropy)

Entropy & IG

$$H(t) = -\sum_k p_k \log_2 p_k$$

$$\text{IG} = H(\text{parent}) - \sum_v \frac{|t_v|}{|t|} H(t_v)$$

Split on feature with highest IG.

5.4 1
Advantages vs Disadvantages

Pros	Cons
Interpretable	High variance (overfits)
No scaling needed	Greedy (not global opt.)
Handles mixed types	Biased to high-cardinality
Non-linear boundaries	Unstable (small data Δ)

5.5 1
Regression Trees

Same splitting logic, but leaves predict the **mean** of training targets. Split criterion: minimize MSE (not Gini/Entropy).

$$\hat{y}_{\text{leaf}} = \frac{1}{|R_m|} \sum_{i \in R_m} y_i$$

Decision Tree (sklearn)

```
from sklearn.tree import (
  DecisionTreeClassifier,
  DecisionTreeRegressor,
  export_text)

dt = DecisionTreeClassifier(
  max_depth=4,
  criterion='gini',  # or 'entropy'
  min_samples_split=5)
dt.fit(X_train, y_train)
print(export_text(dt,
```

```
feature_names=feat_names))
```

6 1

6.1 1

Architectures

Single Cell Perceptron

One neuron:
$z = \sum w_i x_i + b$
$\hat{y} = f(z)$
Binary step function. Only linearly separable problems.

Single Layer Perceptron

One layer of neurons (no hidden layers). Input → Output directly. Still linear.

MLP (Multi-Layer)

Input → Hidden(s) → Output. Non-linear via activations. Universal function approximator.

6.2 1

Backpropagation

1. **Forward pass**: compute predictions $\hat{y}$
2. **Compute loss**: $\mathcal{L}(\hat{y}, y)$
3. **Backward pass**: compute $\frac{\partial \mathcal{L}}{\partial w}$ via chain rule
4. **Update weights**: $w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w}$

Chain Rule: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$

6.3 1

Activation Functions

Common Activations

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$, range $(0, 1)$
Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, range $(-1, 1)$
ReLU: $\max(0, z)$ — avoids vanishing gradient
Leaky ReLU: $\max(0.01z, z)$ — fixes dying ReLU
Softmax: $\frac{e^{z_i}}{\sum_j e^{z_j}}$ — multi-class output

6.4 1

Recurrent Neural Network (RNN)

- Designed for **sequential data** (text, time series)
- Has **hidden state** $h_t = f(W_h h_{t-1} + W_x x_t + b)$
- Output: $y_t = W_y h_t$
- **Problem**: Vanishing gradient over long sequences
- **Solutions**: LSTM, GRU (gating mechanisms)

Simple MLP (PyTorch)

```python
import torch
import torch.nn as nn

class MLP(nn.Module):
  def __init__(self):
    super().__init__()
    self.net = nn.Sequential(
      nn.Linear(784, 256),
      nn.ReLU(),
      nn.Linear(256, 128),
      nn.ReLU(),
      nn.Linear(128, 10))
  def forward(self, x):
    return self.net(x)

model = MLP()
optimizer = torch.optim.Adam(
  model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

7 1

7.1 1

Text Preprocessing & Tokenization

Pipeline:

1. Lowercase, remove punctuation/numbers
2. **Tokenization**: split into words/subwords
3. **Stop-word removal**: remove "the", "is"...
4. **Stemming**: reduce to root (running → run)
5. **Lemmatization**: canonical form (better+am → be)

NLTK Preprocessing

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords

text = "Running faster than ever"
tokens = word_tokenize(text.lower())
stops = set(stopwords.words('english'))
tokens = [t for t in tokens
          if t not in stops]
lem = WordNetLemmatizer()
lemmas = [lem.lemmatize(t) for t in
    tokens]
```

7.2 1

Word Embeddings

Representations

Bag of Words (BoW): Count of words; no order; high-dimensional sparse.

TF-IDF: $\mathrm{TF}(t,d) \times \log \frac{N}{df(t)}$; downweights common words.

Word2Vec: Dense vectors; similar words close in space.
CBOW: context → center word
Skip-gram: center → context words

GloVe: Co-occurrence matrix factorization.

FastText: Subword embeddings; handles OOV.

7.3 1

Sequential Modeling

- **n-gram Models**: Predict next token from previous $n-1$ tokens. Sparsity problem.
- **RNN**: Processes sequence step-by-step; captures order.
- **LSTM**: Long Short-Term Memory; uses *forget, input, output gates*; solves vanishing gradient.
- **GRU**: Simplified LSTM; *reset + update gates*; faster training.

7.4 1

Language Foundations

- **Language Model**: $P(w_1, w_2, \ldots, w_n) = \prod_i P(w_i | w_1, \ldots, w_{i-1})$
- **Perplexity**: Measure of LM quality; lower = better

$$\mathrm{PPL} = P(w_1, \ldots, w_N)^{-1/N}$$

- **Vocabulary**: Set of unique tokens
- **OOV**: Out-of-vocabulary problem; handled by subword tokenization (BPE)

8 1

8.1 1

Transformers

Key Components:

1. **Input Embeddings**: token + **positional encoding** (since no recurrence)
2. **Multi-Head Self-Attention**: attends to all positions simultaneously
3. **Feed-Forward Network**: per-position MLP
4. **Layer Normalization + Residual Connections**
5. **Encoder**: bidirectional context (BERT)
6. **Decoder**: autoregressive generation (GPT)

Self-Attention

Inputs: Query Q, Key K, Value V

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- d_k = key dimension (scaling prevents saturation)
- $Q = XW^Q$, $K = XW^K$, $V = XW^V$
- **Multi-Head**: run h attention heads in parallel, concatenate

Positional Encoding:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

Minimal Self-Attention (NumPy)

```python
import numpy as np

def self_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)          # (seq, seq)
    weights = np.exp(scores - scores.max())   # numerical stability
    weights /= weights.sum(axis=-1, keepdims=True)  # softmax
    return weights @ V                        # (seq, d_v)

# Example
seq_len, d_model = 4, 8
X = np.random.randn(seq_len, d_model)
Wq, Wk, Wv = [np.random.randn(d_model, d_model) for _ in range(3)]
Q, K, V = X @ Wq, X @ Wk, X @ Wv
out = self_attention(Q, K, V)
print("Output shape:", out.shape)  # (4, 8)
```

8.2 1

Ethical Considerations

Responsible AI & Fairness

- **Fairness**: Equal treatment across groups
- **Accountability**: Clear responsibility for decisions
- **Transparency**: Explainability of model
- **Privacy**: Protect user data
- **Safety**: Avoid harm in deployment

Bias Mitigation

Sources of bias:

- Training data bias
- Label bias (human annotators)
- Algorithmic amplification

Mitigation:

- Diverse datasets
- Fairness constraints
- Adversarial debiasing
- Regular audits

Misinformation Prevention

- **Hallucination**: Model generates false but confident info
- **RAG**: Retrieval-Augmented Generation grounds responses
- **Watermarking**: Mark AI-generated content
- **Human-in-loop**: Review critical outputs
- **Fact-checking**: Ground truth verification pipelines

8 1 MCQ

Practice Bank (40 Questions)

Instructions

40 MCQs covering all 8 topics. Answers with explanations at the end.

— Topic 1: ML Foundations —

Q1. In traditional programming, the programmer provides rules and data to get output. In Machine Learning:

(A) Rules and output are given to get data
(B) Data and output are given to learn rules
(C) Data and rules produce the model
(D) Output is given and rules emerge automatically

Q2. Which of the following is the correct hierarchy?

(A) ML $\supset$ DL $\supset$ AI
(B) DL $\supset$ ML $\supset$ AI
(C) AI $\supset$ ML $\supset$ DL
(D) AI $\supset$ DL $\supset$ ML

Q3. A robot learns to walk by receiving a reward when it moves forward and a penalty when it falls. This is:

(A) Supervised Learning
(B) Unsupervised Learning
(C) Reinforcement Learning
(D) Semi-Supervised Learning

Q4. Customer segmentation (grouping similar customers) is best framed as which ML task?

(A) Regression
(B) Classification
(C) Clustering
(D) Ranking

Q5. Predicting tomorrow's stock price (a continuous value) is a:

(A) Classification task
(B) Clustering task
(C) Regression task
(D) Reinforcement task

— Topic 2: Data Preprocessing —

Q6. For a skewed numerical feature, which imputation strategy is most robust?

(A) Mean imputation
(B) Mode imputation
(C) Median imputation
(D) Drop the column

Q7. Min-Max normalization transforms data to the range:

(A) $(-\infty, +\infty)$
(B) $(-1, 1)$
(C) $(0, 1)$
(D) $(0, +\infty)$

Q8. Z-score standardization produces output with:

(A) Mean=1, Std=0
(B) Mean=0, Std=1
(C) Mean=0.5, Std=0.5
(D) Values between 0 and 1

Q9. One-Hot Encoding is preferred over Label Encoding when:

(A) Categories have a natural order
(B) There are many categories (>100)
(C) Categories have no ordinal relationship
(D) The feature is numerical

Q10. TF-IDF stands for:

(A) Text Frequency – Inverse Document Frequency
(B) Term Frequency – Inverse Document Frequency
(C) Token Frequency – Index Document Frequency
(D) Term Frequency – Internal Data Feature

— Topic 3: Linear & Logistic Regression —

Q11. The cost function minimized by Linear Regression is:

(A) Cross-entropy loss
(B) Mean Absolute Error
(C) Mean Squared Error
(D) Hinge loss

Q12. Which assumption of Linear Regression states that the variance of residuals is constant?

(A) Normality
(B) Linearity
(C) Homoscedasticity
(D) Independence

Q13. The sigmoid function $\sigma(0)$ equals:

(A) 0
(B) 1
(C) 0.5
(D) -1

Q14. Logistic Regression uses which loss function?

(A) MSE
(B) Binary Cross-Entropy
(C) Hinge Loss
(D) MAE

Q15. What type of decision boundary can Logistic Regression produce?

(A) Any non-linear boundary
(B) Circular boundaries
(C) Only linear boundaries
(D) Tree-based boundaries

— Topic 4: Model Evaluation —

Q16. A model has high training accuracy but low test accuracy. It is:

(A) Underfitting (high bias)
(B) Overfitting (high variance)
(C) A perfect model
(D) Experiencing data leakage only

Q17. You are building a fraud detection system. Missing actual frauds is very costly. Which metric should you prioritize?

(A) Precision
(B) Accuracy
(C) Recall
(D) Specificity

Q18. A spam filter should avoid flagging legitimate emails as spam. Which metric matters most?

(A) Recall
(B) Precision
(C) F1 Score
(D) RMSE

Q19. RMSE is preferred over MAE when:

(A) Outliers should be ignored
(B) Large errors should be penalized more
(C) The data has no outliers
(D) Speed of computation matters

Q20. In k-Fold Cross-Validation with k=5, how many times is the model trained?

(A) 1
(B) 4
(C) 5
(D) 10

Q21. An R^2 value of 0.85 means:

(A) Model is 85% accurate
(B) Model explains 85% of variance in y
(C) Error is 15%
(D) Model predicts 85% of samples correctly

— Topic 5: Decision Trees —

Q22. A Decision Tree node with all samples from the same class has Gini impurity of:

(A) 0.5
(B) 1.0
(C) 0.0
(D) Cannot be determined

Q23. Information Gain is calculated as:

(A) Entropy of child – Entropy of parent
(B) Gini of parent – Gini of children
(C) Entropy of parent – weighted Entropy of children
(D) MSE of parent – MSE of children

Q24. Which of the following is NOT an advantage of Decision Trees?

(A) Easy to interpret
(B) No need for feature scaling
(C) Robust to overfitting
(D) Can handle non-linear data

Q25. In a Regression Tree, each leaf node outputs the:

(A) Majority class of training samples
(B) Median of training targets
(C) Mean of training targets
(D) Mode of training targets

— Topic 6: Neural Networks —

Q26. Which architecture can solve the XOR problem that a Single-Layer Perceptron cannot?

(A) Single Cell Perceptron
(B) Multi-Layer Perceptron
(C) Linear Regression
(D) Logistic Regression

Q27. ReLU activation is defined as:

(A) $\sigma(x) = 1/(1 + e^{-x})$
(B) $\tanh(x)$
(C) $\max(0, x)$
(D) $e^x / \sum e^{x_i}$

Q28. The vanishing gradient problem in deep networks is most severe with which activation?

(A) ReLU
(B) Sigmoid
(C) Leaky ReLU
(D) Linear

Q29. In backpropagation, which rule is used to compute gradients layer by layer?

(A) Product rule
(B) Chain rule
(C) Quotient rule
(D) L'Hopital's rule

Q30. RNNs maintain memory through:

(A) Attention weights
(B) Convolutional filters
(C) Hidden state passed between time steps
(D) Residual connections

Q31. Softmax is used in the output layer for:

(A) Binary classification
(B) Regression
(C) Multi-class classification
(D) Dimensionality reduction

— Topic 7: NLP —

Q32. Which operation is NOT part of standard NLP text preprocessing?

(A) Tokenization
(B) Stopword removal
(C) Gradient descent
(D) Lemmatization

Q33. Word2Vec's Skip-gram model predicts:

(A) Center word from surrounding context
(B) Surrounding context words from center word
(C) Sentiment from word sequence
(D) Next sentence from current one

Q34. LSTM gates that decides what past information to discard is:

(A) Output gate
(B) Input gate
(C) Forget gate
(D) Reset gate

Q35. Perplexity as a metric for language models: lower perplexity means:

(A) Model is more confused
(B) Model is better / more certain
(C) Model overfits more
(D) Model uses fewer parameters

— Topic 8: LLM & Responsible AI —

Q36. In the Transformer self-attention formula $\text{softmax}(QK^T/\sqrt{d_k})V$, why is $\sqrt{d_k}$ used?

(A) To increase gradient flow
(B) To prevent dot products from becoming too large, causing softmax saturation
(C) To normalize the value vectors
(D) To add positional information

Q37. BERT uses which type of Transformer architecture?

(A) Decoder-only
(B) Encoder-only
(C) Encoder-Decoder
(D) Single-head attention only

Q38. GPT-style language models generate text using:

(A) Bidirectional context (all tokens)
(B) Only future tokens
(C) Autoregressive decoding (left-to-right)
(D) Masked multi-head attention in encoder

Q39. When an LLM confidently generates factually incorrect information, it is called:

(A) Overfitting
(B) Hallucination
(C) Data poisoning
(D) Adversarial attack

Q40. Retrieval-Augmented Generation (RAG) is used primarily to:

(A) Make models faster
(B) Reduce training data requirements
(C) Ground model responses in retrieved factual documents
(D) Improve attention efficiency

Answer Key & Explanations

Q1. B — Data + outcomes → rules is the ML paradigm.
Q2. C — AI ⊃ ML ⊃ DL is the standard hierarchy.
Q3. C — Reward/penalty = Reinforcement Learning.
Q4. C — Grouping unlabeled data = Clustering.
Q5. C — Continuous value prediction = Regression.

Q6. C — Median is robust to skew/outliers.
Q7. C — Min-Max $\in [0, 1]$.
Q8. B — Z-score → mean=0, std=1.
Q9. C — No ordinal relation → One-Hot.
Q10. B — Term Frequency – Inverse Document Frequency.

Q11. C — Linear Regression minimizes MSE.
Q12. C — Constant variance = Homoscedasticity.
Q13. C — $\sigma(0) = 1/(1 + 1) = 0.5$.
Q14. B — Binary cross-entropy / log-loss.
Q15. C — Logistic Regression is linear.

Q16. B — High train, low test → overfitting.
Q17. C — Missing frauds (FN) is costly → maximize Recall.
Q18. B — FP costly for spam → maximize Precision.
Q19. B — RMSE penalizes large errors (squared).
Q20. C — k=5 means 5 train/test iterations.
Q21. B — $R^2 = 0.85 \rightarrow 85\%$ variance explained.

Q22. C — Pure node → Gini = 0.
Q23. C — IG = Parent entropy – Weighted child entropy.
Q24. C — DTs overfit; robustness is a disadvantage.
Q25. C — Regression tree leaf = mean of targets.

Q26. B — MLP with hidden layers solves XOR.
Q27. C — ReLU = $\max(0, x)$.
Q28. B — Sigmoid's gradient ≈ 0 for large $|x|$.
Q29. B — Chain rule enables layer-wise gradient propagation.
Q30. C — Hidden state h_t carries sequence memory.
Q31. C — Softmax for multi-class output probabilities.

Q32. C — Gradient descent is optimization, not NLP preprocessing.
Q33. B — Skip-gram: center word → predict context.
Q34. C — Forget gate discards irrelevant past info.
Q35. B — Lower PPL = model is more confident/better.

Q36. B — Scaling prevents softmax saturation.
Q37. B — BERT = encoder-only (bidirectional).
Q38. C — GPT generates tokens left-to-right.
Q39. B — Confident wrong output = hallucination.
Q40. C — RAG grounds responses in real documents.

Last-Minute Cheat Sheet

Sigmoid range: $(0, 1)$
Tanh range: $(-1, 1)$
ReLU: $\max(0, x)$
Softmax: multi-class

Gini pure node: 0
Gini max (binary): 0.5
Best IG split: highest IG

Recall: TP/(TP+FN)
Precision: TP/(TP+FP)
F1: harmonic mean P&R

High bias: underfit
High variance: overfit
k-Fold: k train iterations

Z-score: mean=0, std=1
Min-Max: [0,1]
Median: skewed data

LR boundary: linear only
MLP: universal approx.
RNN hidden state: memory

Attention: $\text{softmax}(QK^T/\sqrt{d_k})V$
BERT: encoder, bidirectional
GPT: decoder, autoregressive
RAG: grounds LLM facts
Hallucination: confident errors

TF-IDF: term freq × inv-doc freq
Word2Vec CBOW: context→word
Skip-gram: word→context
Forget gate: LSTM
PPL: lower = better LM